

WHAT IS ANDROID?

Android is a mobile operating system that is based on a modified version of Linux. It was originally developed by a startup of the same name, Android, Inc. In 2005, as part of its strategy to enter the mobile space, Google purchased Android, Inc. and took over its development work (as well as its development team).

Google wanted the Android OS to be open and free, so most of the Android code was released under the open source Apache License. That means anyone who wants to use Android can do so by downloading the full Android source code. Moreover, vendors (typically hardware manufacturers) can add their own proprietary extensions to Android and customize Android to differentiate their products from others. This development model makes Android very attractive to vendors, especially those companies affected by the phenomenon of Apple's iPhone, which was a hugely successful product that revolutionized the smartphone industry. When the iPhone was launched, many smartphone manufacturers had to scramble to find new ways of revitalizing their products. These manufacturers saw Android as a solution, meaning they will continue to design their own hardware and use Android as the operating system that powers it. Some companies that have taken advantage of Android's open source policy include Motorola and Sony Ericsson, which have been developing their own mobile operating systems for many years.

The main advantage to adopting Android is that it offers a unified approach to application development. Developers need only develop for Android in general, and their applications should be able to run on numerous different devices, as long as the devices are powered using Android. In the world of smartphones, applications are the most important part of the success chain.

Android Versions

Android has gone through quite a number of updates since its first release. Table 1-1 shows the various versions of Android and their codenames.

TABLE 1-1: A Brief History of Android Versions

ANDROID VERSION	RELEASE DATE	CODENAME
1.1	February 9, 2009	
1.5	April 30, 2009	Cupcake
1.6	September 15, 2009	Donut
2.0/2.1	October 26, 2009	Éclair
2.2	May 20, 2010	Froyo
2.3	December 6, 2010	Gingerbread
3.0/3.1/3.2	February 22, 2011	Honeycomb
4.0	October 18, 2011	Ice Cream Sandwich

WOW! eBook
www.wowebook.org

ANDROID VERSION	RELEASE DATE	CODENAME
4.1	July 9, 2012	Jelly Bean
4.4	October 31, 2013	KitKat
5.0	November 12, 2014	Lollipop
6.0	October 5, 2015	Marshmallow
7.0	TBD	Nougat

In 2016, Google released Android 7.0; the following are the key changes in Android 7.0:

- Split-screen multi-window mode
- Redesigned notification shade
- Refined “Doze” feature
- Switch from JRE (Java Runtime Environment) to OpenJDK

One important thing to keep in mind as you are looking at Android versions is that each version has its own features and APIs (application programming interfaces). Therefore, if your application is written for the newest version of Android, and it uses an API that was not present in an older version of Android, then only devices running that newer version of Android will be able to use your application.

Features of Android

Because Android is open source and freely available to manufacturers for customization, there are no fixed hardware or software configurations. However, the base Android OS supports many features, including

- **Storage**—SQLite, a lightweight relational database, for data storage. Chapter 7 discusses data storage in more detail.
- **Connectivity**—GSM/EDGE, IDEN, CDMA, EV-DO, UMTS, Bluetooth (includes A2DP and AVRCP), Wi-Fi, LTE, and WiMAX. Chapter 11 discusses networking in more detail.
- **Messaging**—Both SMS and MMS. Chapter 9 discusses messaging in more detail.
- **Media support**—H.263, H.264 (in 3GP or MP4 container), MPEG-4 SP, AMR, AMR-WB (in 3GP container), AAC, HE-AAC (in MP4 or 3GP container), MP3, MIDI, Ogg Vorbis, WAV, JPEG, PNG, GIF, and BMP.
- **Hardware support**—Accelerometer sensor, camera, digital compass, proximity sensor, and GPS.
- **Multi-touch**—Multi-touch screens.
- **Multi-tasking**—Multi-tasking applications.
- **Tethering**—Sharing of Internet connections as a wired/wireless hotspot.

Android's web browser is based on the open source WebKit and Chrome's V8 JavaScript engine.

Architecture of Android

To understand how Android works, take a look at Figure 1-1, which shows the various layers that make up the Android operating system (OS).

The Android OS is roughly divided into five sections in four main layers:

- **Linux kernel**—This is the kernel on which Android is based. This layer contains all the low-level device drivers for the various hardware components of an Android device.
- **Libraries**—These contain the code that provides the main features of an Android OS. For example, the SQLite library provides database support so that an application can use it for data storage. The WebKit library provides functionalities for web browsing.
- **Android runtime**—The Android runtime is located in the same layer with the libraries and provides a set of core libraries that enable developers to write Android apps using the Java programming language. The Android runtime also includes the Dalvik virtual machine, which enables every Android application to run in its own process, with its own instance of the Dalvik virtual machine. (Android applications are compiled into Dalvik executables). Dalvik is a specialized virtual machine designed specifically for Android and optimized for battery-powered mobile devices with limited memory and CPU power.
- **Application framework**—The application framework exposes the various capabilities of the Android OS to application developers so that they can make use of them in their applications.
- **Applications**—At this top layer are the applications that ship with the Android device (such as Phone, Contacts, Browser, and so on), as well as applications that you download and install from the Android Market. Any applications that you write are located at this layer.

Android Devices in the Market

Android devices come in all shapes and sizes including, but not limited to, the following types of devices:

- Smartphones
- Tablets
- E-reader devices
- Internet TVs
- Automobiles
- Smartwatches

Chances are good that you own at least one of the preceding devices. Figure 1-2 shows the Samsung Galaxy Edge 7.

WOW! eBook
www.wowebook.org

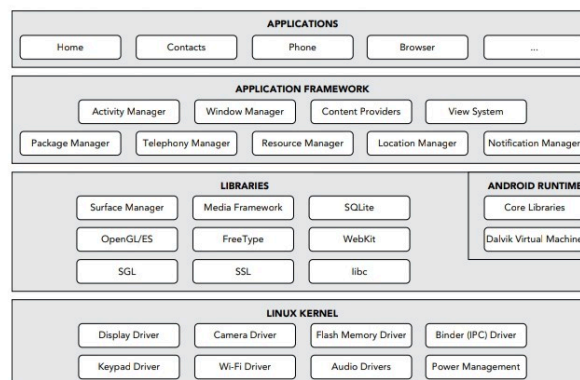


FIGURE 1-1

Location-Based Services

WHAT YOU WILL LEARN IN THIS CHAPTER

- Displaying Google Maps in your Android application
- Displaying zoom controls on the map
- Switching between the different map views
- Retrieving the address location touched on the map
- Performing geocoding and reverse geocoding
- Obtaining geographical data using GPS, Cell-ID, and Wi-Fi triangulation
- Monitoring for a location
- Building a location tracker application

CODE DOWNLOAD The wrox.com code downloads for this chapter are found at www.wrox.com/go/beginningandroidprog on the Download Code tab. The code is in the chapter 10 download and individually named according to the names throughout the chapter.

You have seen the explosive growth of mobile apps in recent years. One category of apps that is very popular is Location-Based Services, commonly known as LBS. LBS apps track your location, and might offer additional services such as locating amenities nearby, offering

WOW! eBook
www.wowebook.org

suggestions for route planning, and so on. Of course, one of the key ingredients in an LBS app is maps, which present a visual representation of your location.

This chapter shows you how to make use of Google Maps in your Android application, as well as how to manipulate the map view programmatically. In addition, you find out how to obtain your geographical location using the `LocationManager` class available in the Android SDK. This chapter ends with a project to build a location tracker application that you can install on an Android device and use to track the location of friends and relatives using SMS messaging.

DISPLAYING MAPS

Google Maps is one of the many applications bundled with the Android platform. In addition to simply using the Maps application, you can also embed it into your own applications and make it do some very cool things. This section describes how to use Google Maps in your Android applications and programmatically perform the following:

- Change the views of Google Maps.
- Obtain the latitude and longitude of locations in Google Maps.
- Perform geocoding and reverse geocoding (translating an address to latitude and longitude and vice versa).

Creating the Project

To get started, you need to first create an Android project so that you can display Google Maps in your activity:

1. Using Android Studio, create an Android project and name it LBS.
2. From the Create New Project Wizard, select Google Maps Activity as shown in Figure 10-1.

Obtaining the Maps API Key

Beginning with the Android SDK release v1.0, you need to apply for a free Google Maps API key before you can integrate Google Maps into your Android application. When you apply for the key, you must also agree to Google's terms of use, so be sure to read them carefully.

To get a Google Maps key, open the `google_maps_api.xml` file that was created in your LBS project. Within this file is a link to create a new Google Maps key. Simply copy and paste the link into your browser and follow the instructions. Make note of the key that Google gives you because you need it later in this project.

DISPLAYING NOTIFICATIONS

So far, you have been using the `Toast` class to display messages to the user. While the `Toast` class is a handy way to show users alerts, it is not persistent. It flashes on the screen for a few seconds and then disappears. If it contains important information, users may easily miss it if they are not looking at the screen.

For messages that are important, you should use a more persistent method. In this case, you should use the `NotificationManager` to display a persistent message at the top of the device, commonly known as the *status bar* (sometimes also referred to as the *notification bar*). The following Try It Out demonstrates how.

TRY IT OUT Displaying Notifications on the Status Bar (Notifications.zip)

```
} 2
```

DatePicker View

Another view that is similar to the `TimePicker` is the `DatePicker`. Using the `DatePicker`, you can enable users to select a particular date on the activity. The following Try It Out shows you how to use the `DatePicker`.

TRY IT OUT Using the `DatePicker` View

USING LIST VIEWS TO DISPLAY LONG LISTS

List views are views that enable you to display a long list of items. In Android, there are two types of list views: `ListView` and `SpinnerView`. Both are useful for displaying long lists of items. The Try It Outs in this section show them in action.

ListView View

The `ListView` displays a list of items in a vertically scrolling list. The following Try It Out demonstrates how to display a list of items using the `ListView`.

TRY IT OUT Displaying a Long List of Items Using the `ListView` (BasicViews5.zip)

```
} //...
```

USING WEBVIEW

Aside from the standard views that you have seen up to this point, the Android SDK provides some additional views that make your applications much more interesting. This section explains more about the `WebView`.

WebView

The `WebView` enables you to embed a web browser in your activity. This is very useful if your application needs to embed some web content, such as maps from some other providers, and so on. The following Try It Out shows how you can programmatically load the content of a web page and display it in your activity.

TRY IT OUT Using the `WebView` View (`WebView.zip`)

If you compare the `onClick` approach to the callback method discussed previously, you'll see this is much simpler. Which method you use is really up to you, but this book mostly uses the `onClick` approach.

ProgressBar View

The `ProgressBar` view provides visual feedback about some ongoing tasks, such as when you are performing a task in the background. For example, you might be downloading some data from the web and need to update the user about the status of the download. In this case, the `ProgressBar` view is a good choice. The following activity demonstrates how to use the `ProgressBar` view.

TRY IT OUT Using the ProgressBar View (BasicViews2.zip)

1. Using Android Studio, create an Android project and name it **BasicViews2**.

Context Menu

The previous section showed how the options menu is displayed when the user presses the Menu button. In addition to the options menu, you can also display a context menu. A context menu is usually associated with a view on an activity. A context menu is displayed when the user taps and holds an item. For example, if the user taps a `Button` view and holds it for a few seconds, a context menu can be displayed.

Google Maps is one of the many applications bundled with the Android platform. In addition to simply using the Maps application, you can also embed it into your own applications and make it do some very cool things. This section describes how to use Google Maps in your Android applications and programmatically perform the following:

- Change the views of Google Maps.
- Obtain the latitude and longitude of locations in Google Maps.
- Perform geocoding and reverse geocoding (translating an address to latitude and longitude and vice versa).

Creating the Project

To get started, you need to first create an Android project so that you can display Google Maps in your activity:

1. Using Android Studio, create an Android project and name it LBS.
2. From the Create New Project Wizard, select Google Maps Activity as shown in Figure 10-1.

Obtaining the Maps API Key

Beginning with the Android SDK release v1.0, you need to apply for a free Google Maps API key before you can integrate Google Maps into your Android application. When you apply for the key, you must also agree to Google's terms of use, so be sure to read them carefully.

To get a Google Maps key, open the `google_maps_api.xml` file that was created in your LBS project. Within this file is a link to create a new Google Maps key. Simply copy and paste the link into your browser and follow the instructions. Make note of the key that Google gives you because you need it later in this project.